CS4850 (01), Spring 2026

SP-14 Green Chess AI

Brett Talley, Kenneth Burke, Marc-Henry Mois, Vladimir Ryzhov

Professor Sharon Perry, 5/4/2026

https://ourchessai.github.io/OurChessProject/

https://github.com/OurChessAi/OurChessProject

STATS AND STATUS		
Lines of Code (LOC)	1283	
Components/Tools	Python 3.13, Pygame	
Hours Estimate	470	
Hours Actual	512	
Status	100% Completed	

Contents

1. Introduction 4

2. Requirements 5

2..1. Overview 5

2..2. Functional Requirements 6

2..3. External Interface Requirements 6

3. Analysis and Design 7

3..1. Design Considerations 7

3..2. Architectural Strategies 8

3..3. System Architecture 9

3..4. Detailed System Design 10

4. Development 11

4..1. Technical Detail Summary 11

4..2. Development 13

5. Test Plan and Report 14

5..1. Overview 14

5..2. Testing Summary 15

5..3. Analysis of Scope and Test Focus Areas 15

5..4. Progression Test Objectives 16

5..5. Regression Test Objectives 16

5..6. Test Strategy 17

5..7. Test Type & Approach 17

5..8. Test Execution Schedule 17

5..9. Facility, data, and resource provision plan 17

5..10. Testing Tools 18

5..11. Testing Metrics 18

5..12. Test Environment Plan 18

5..13. Test Environment Details 19

5..14. Establishing Environment 19

5..15. Environment Roles and Responsibilities 19

5..16. Assumptions and Dependencies 19

5..17. Entry and Exit Criteria 20

5..18. Test Milestones and Schedule 21

5..19. Test Report 22

5..20. References 22

6. Version Control 23

7. Conclusion 24

1. Introduction

Our project was to create a functioning chess game, which has 2 game modes: Player vs Player and Player vs AI. The chess game needs to follow all the standard chess rules, including the standard piece movement, castling, promotions, and en passant.

To complete it, we performed these actions, we have set up a version control via Github. We have divided responsibilities between all group members. We have also met on a weekly basis in order to update the progression of these tasks, as well as assign new responsibilities.

2. Requirements

2..1. Overview

Create a self-playing AI Chess Engine that offers both player vs player and player vs AI game modes. You may use any technology.

Objective:

Create a working chess game which follows the standard rules of chess. The chess has to have two game modes: Player vs Player (PvP) and Player vs Artificial Intelligence (PvAI). In case of abundance of time, we might decide to pursue the objective of adjustable difficulty for the AI game mode.

Code in Python using Pygame Module.

2..1.1. Project Scope

Working chess, meaning:

- No illegal moves
- En passant, castling

PvP chess

- White cannot move black's pieces and vice versa
- Possibly board will flip every move

PvAI chess

- Engine automatically plays a move when player makes a move

2..1.2. Assumptions

This will not be an online game; we will be developing the PvP section under the assumption that both players are using the same computer.

2..1.3. References

https://www.youtube.com/watch?v=OpL0Gcfn4B4

- Am informative guide for game design, the board, and the pieces

https://www.freecodecamp.org/news/simple-chess-ai-step-by-step-1d55a9266977/

- Creation of a simple chess ai

https://chessboardjs.com/

- Easy embedment of a chess board onto a website

2..2. Functional Requirements

2..2.1. General Requirements

- Pawn must be able to move forward two squares on first move.
    - Down the board for black, up the board for white.
    - Pawns must not be able to move backwards.
    - Pawn captures must only be available on the immediate diagonal in their respective direction.
    - En passant captures must be available
- Rooks must be able to move horizontally and vertically along the board.
- Bishops must be restricted to their diagonal.
- Knights must be able to move in offsets of 2 and 1 in an L shape and not be restricted by pieces in their way.
- Queen must be able to have the movement of both the rook and the bishop;
    - Move along the diagonals and the straight lines.
- King must be able to move one square in any direction.
    - King must not move into a square where it can be captured.
- Pieces should not be able to move if it exposes the king to be captured
    - Enforce pins, don't let the king move into an attacked square.
- Castling should be properly implemented.
    - No castling out of check.
    - No castling if the squares in between the king and rook are attacked.
    - No castling if the squares in between have pieces residing on them.

2..2.2. PvP Requirements

- 2 players
- Can only move their respective pieces
    - When it is white's turn to move, black cannot make a move and vice versa.

2..2.3. PvAI Requirements

- Engine automatically generates the best move
- If time allows it, rank the moves
- Ranking the moves allows for adjustable difficulty

2..3. External Interface Requirements

2..3.1. User Interface Requirements

Mandatory:

- Menu with choice of PvP and PvAI
- Chess pieces graphics

Optional (complete if there is enough time):

- Show legal moves on piece clicked
- Choice of difficulty vs AI
- Option to resign/draw

3. Analysis and Design

3..1. Design Considerations

3..1.1. Assumptions and Dependencies

- Python with the Pygame module will be used to develop the software, the use of the software will be accessible through an executable.
- The software will be available for use on both Windows and UNIX operating systems.
- End user will only need knowledge on how to play chess. If they decide to play PvP chess, it is assumed that they are playing locally on the same system. If they decide to play against the AI, they will be able to make their move and the AI will respond with a move of it's own.
- Possible changes in functionality may include an option for elo adjustment against the AI.

3..1.2. General Constraints

No limitations exist across hardware, software, and end user environments. Data repository will be kept locally within the system. As per performance, the AI must make their moves relatively quickly.

- Other requirements:

  General Requirements:

  - No illegal moves, no phasing through pieces, etc.
  - En passant, castling

  PvP Requirements:

  - 2 players
  - Can only move their respective pieces

  PvAI Requirements:

  - Engine automatically generates the best move
  - If time allows it, rank the moves
  - Ranking the moves allows for adjustable difficulty

3..1.3. Development Methods

The development of the software product will follow an iterative methodology. This means that we will begin by simply making the game of chess playable, and then iterate by adding full move legality, local PvP play, and an AI opponent. Software will be developed in Python with the Pygame module, but distributed in the form of an executable so end users need not know how to program to use the software.

The software will support Windows as well as UNIX-based operating systems. More rigid methodologies like Waterfall were evaluated but will not be used due to the gameplay and interaction requirements changing over the course of development. The AI opponent will follow a standard minimax algorithm with alpha–beta pruning, and will make a move in response to each of the player's moves. End users will only need to know how to play chess. PvP mode will assume two players are playing on the same computer locally.

3..2. Architectural Strategies

Architecturally, the system will include one shared chess core (board state, rules, movement validation, etc.) for use by both local PvP and AI chess play. Doing this should enforce shared behavior and prevent duplicated logic between the two approaches. Implementation will be in Python via Pygame. This allows for quick iteration and easy packaging into an executable, at the cost of advanced graphics and peak performance. Support platforms will be Windows and UNIX-style operating systems.

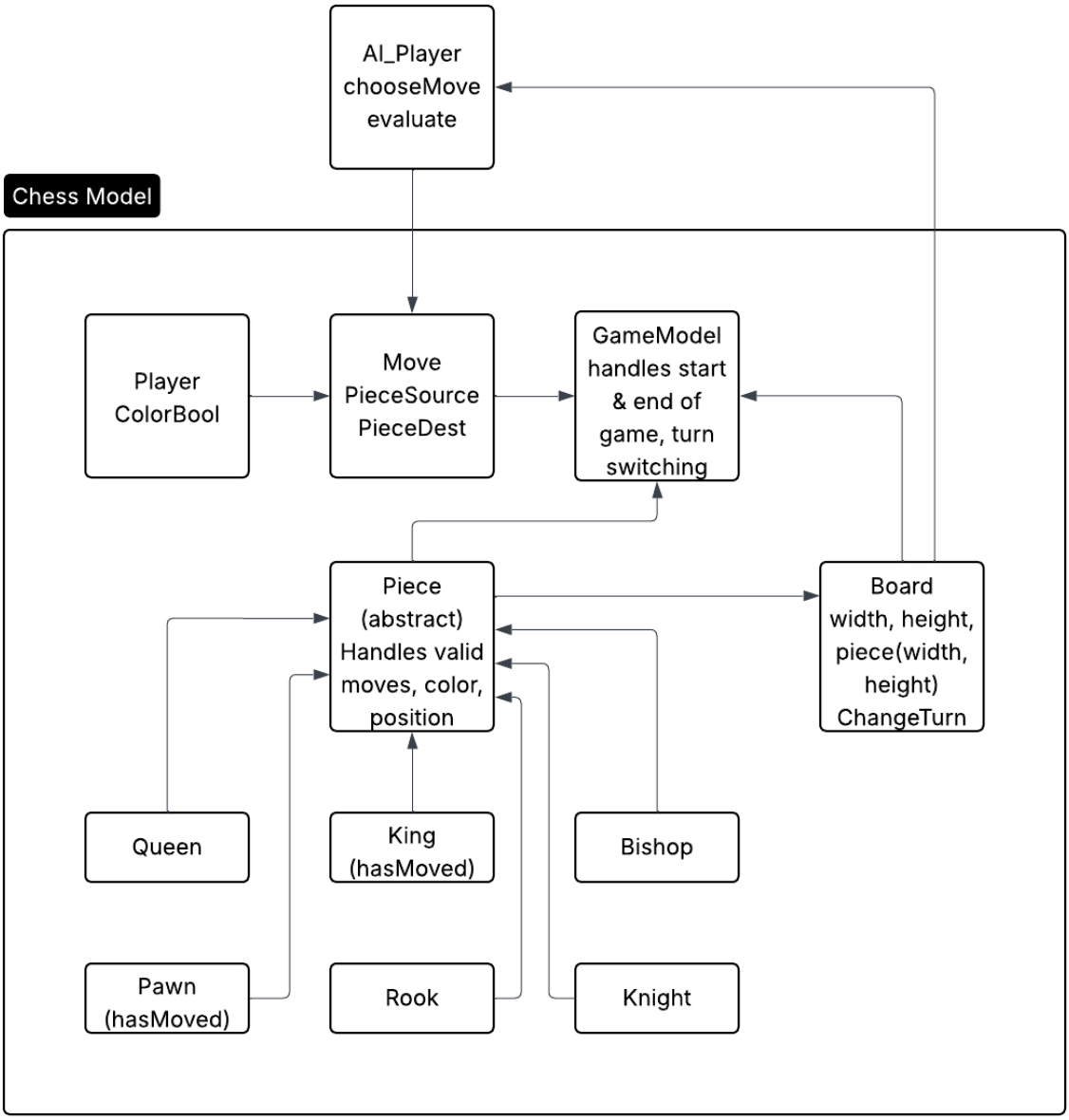
UI will be based on click-to-move actions. This was selected due to its straightforward nature and minimal learning curve. AI will be swappable modules that conform to a standard interface. Standard minimax with alpha–beta pruning will be implemented, allowing general difficulty adjustment and easier swapping of a better AI later. Invalid actions will simply be rejected with informative text (versus trying to recover from invalid actions). Gameplay will be strictly single player versus and local. No networking, threads, or database support will be added. This avoids complexity in other areas.

For expansion, the goal is to only add modules that extend behavior in supported ways. Examples include support for stronger AI, optional save/load, or UI polish.

3..3. System Architecture

At the heart of the system is a Chess Model object that knows about all of the game logic/state. This ensures that there is only one authoritative game state available to all of the other objects interacting with the game. GameModel encapsulates logic about how a game starts, ends, and switches turns. Board maintains the board array, its dimensions, and which side's turn it is.

Players, whether they be human or AI issue a Move (fromPiece, toPosition). The AIPlayer looks at the current Board state and decides upon a Move based on its internal decision making logic. The Move is then passed into the same Move execution workflow as if a human player had issued the move. Validation logic was broken out into an abstract Piece class. The subclasses (Pawn, Rook, Knight, Bishop, Queen, King) are then responsible for how they can move based on their type as well as specifying any special rules (ex. hasMoved).



3..4. Detailed System Design

3..4.1. Classification

The system will be structured as a single desktop application compiled into a single executable file. Within the application there will be multiple cooperating classes that model the game logic, board state, player IO, and AI.

3..4.2. Definition

This system will allow full chess gameplay, enabling local PvP as well as versus computer play. The system will control game flow, game board, movement validation, and AI calculations and will enforce all normal rules of chess.

3..4.3. Constraints

The system will run locally on one machine and will not involve networking or external databases. Moves should be checked before they are performed and illegal moves will be rejected safely. The system should never hang while a game is in progress. AI time should be limited as necessary to ensure responsiveness.

3..4.4. Resources

A description of any and all resources that are managed, affected, or needed by this entity. Resources are entities external to the design such as memory, processors, printers, databases, or a software library. This should include a discussion of any possible race conditions and/or deadlock situations, and how they might be resolved.

3..4.5. Interface

The system will present a user-interface for player interactions. Players will be able to select pieces, move them, and receive updates from the game. Behind the scenes, the system will expose services that allow moves to start games, make moves, validate them against rules, and trigger AI play.

4. Development

4..1. Technical Detail Summary

4..1.1. Chessboard

Dimensions.py has fixed board dimension values, 100x100 pixels a square .

The board is initialized as a 2D array to represent each square. It then places pictures of pieces in their designated starting positions.

I/O features:

- Square is highlighted on hover
- Legal moves are shown when dragging initiates

4..1.2. Chess game engine

Main.class runs the main game loop. It listens for mouse and keyboard events, redraws the screen, and runs the game logic once per frame.

Jeu.class tracks the current status of the game. It stores the current player in **next_player**. It knows about the board state, it passes events to Drag if a piece is being dragged, it displays valid moves. It highlights the last move made. When a piece is successfully moved, it calls **next_turn()** to change whose turn it is.

Board.class actually implements all of the movement logic. It stores the board itself in **self.squares**. It positions all the pieces at game start. It knows how to calculate legal moves for each piece. It can validate if a move is legal or not. It updates pieces' positions on the board. It takes care of removing captured pieces from the board. It deals with pawn promotion, castling, and en passant. It remembers the last move that was made.

Piece.class, Pawn.class, Knight.class, Bishop.class, Rook.class, Queen.class, and King.class represent each of the chess pieces. Each piece knows its color, whether it has moved yet or not, and a list of its current legal moves. The engine calculates those legal moves.

Move.class defines a move as having a starting square and an ending square.

Drag.class deals with player interaction when a piece is selected and dragged around the screen.

When the program runs:

Main.mainloop() calls its **run()** method, which runs infinitely inside of a while True:. While that loop is running, the game board is alive. Inside of that loop, the board is drawn, pieces are drawn, valid moves are highlighted, mouse movements are checked, and mouse button clicks/releases are checked.

If the player clicks on a square, **board.square_select(row, col)** determines if there is a piece on that square. If there is, and that piece belongs to the player whose turn it is, **board.moves_calc(piece, row, col, bool=True)** calculates all the piece's legal moves.

If the player clicks and drags the piece to a different square, **Move(move_row, move_col, piece.row, piece.col)** is created, identifying the piece's starting square and ending square. **board.valid_move(piece, move)** is then called to determine if the ending square is in the piece's list of legal moves.

If the move is legal, **board.move(piece, move)** will move the piece. It edits the board so that the piece is removed from its old square and added to its new square. If necessary, it will also remove captured pieces, and deal with en passant, pawn promotion, or castling. Finally, it will set **last_move** to the moved moved, erase the old valid moves, set the piece moved attribute to True, and **game.next_turn()** will set **next_player** to the other player.

4..1.3. Pieces – rules

- Pawn
    - Has a property direction to differentiate between white and black moves. An if statement is included for the first pawn move being able to move 2 squares. Conditions are included for an opponent piece being diagonally in front of the pawn, as well as being blocked from the front.
    - When a pawn is pushed two spaces, the space behind the pushed pawn becomes capturable for the next move. This enables the player to capture the pawn en passant, if their own pawn is available to execute the move.
- Rook, Bishop, and Queen all use the **straight_moves** method, and the **straight_moves** method iterates for all possible squares in the parameterized direction. It breaks and stops iterating once the edge of the board is reached or if there is a piece in the way (breaks on the square of the opponent piece, breaks on the square before your own piece).
    - Rook utilizes the straight moves along the rows and columns.
    - Bishop utilizes the straight moves along the diagonals.
    - Queen utilizes the straight moves of all eight directions.
- Knight has eight possible moves to make. For each possible move it checks if the square is a legal square to land on.
- King has the ability to move one square in any direction
    - Since each piece has a ‘moved’ property, an unmoved king and the unmoved rook on it’s castling side has the ability to castle by altering the column position of both pieces.

4..1.4. Modes of play

- PvP is implemented as a local PvP; in other words, 2 people will sit at one computer and alternate taking moves
- PvAI will have an algorithm-based program playing against the player.

4..1.5. AI structure and logic

- The evaluation algorithm is a negamax with alpha beta pruning.
- Pieces are assigned a constant value that corresponds to how valuable the piece is.
- A constant piece table is assigned to calculate the value of the piece's position on the board.
- Evaluation of the position is calculated by the value of the piece with it's corresponding position in it's table matrix. White pieces add to the evaluation while black pieces subtract from the evaluation.
- Negamax recursively searches the best position while ignoring the worst positions. The best score of the position is kept as the alpha value, and any position that is worse than the alpha value gets beta pruned.
    - When possible moves are searched, the order in which they will be searched in is checks, captures, and attacks since those moves have a higher chance of producing a desirable outcome.
    - The highest four resulting evaluations are explored, leading to more possible better evaluations after making an initial 'poor' move.

4..2. Development

During the development our team was running Python 3.13, but the program is likely to also work with relatively recent older versions.

Pygame is an essential module which is required to run the program.

Installation:

git clone https://github.com/OurChessAi/OurChessProject/

cd SP14-ChessAIGreen/SRC

python main.py

4..2.1. Challenges During Development

- Issues with the AI Depth
    - Due to deep copy, we move generation took a very long time and a reasonable depth could not be used without the AI making a move in a reasonable amount of time.
- Issues with boundaries
    - Game would crash if a piece was dragged outside of the board
- Issues with premature checkmating
    - Checkmate would be delivered even when the opponent had legal moves
- Issues with piece logic
    - Pieces were able to phase through other pieces
    - This was an interesting one to handle since piece phasing happened only under certain conditions that could not be defined.

5. Test Plan and Report

5..1. Overview

5..1.1. Scope

The scope of this section is to define what will be tested and how, as well as to document the test and the results of the tests. The document will define:

- What was tested
- How it was tested
- Results of the test and what changes are planned according to the results

5..2. Testing Summary

5..2.1. Scope of Testing

5..2.1.1. In scope

- What will be tested: crashing, illegal move (phasing), premature checkmate, issues with AI (static AI, major AI slowdown as depth increases, etc.)
- How testing will be performed:
    - By hand; testers will take previously discovered issues and see if those are still present, as well as try to push the program further to see if anything else breaks.
    - AI will be tested for being static by playing against it; the depth slowdown will require changing and testing the AI code.

5..3. Analysis of Scope and Test Focus Areas

5..3.1. Regression Testing

Test case for game launching properly

Test case for piece boundaries off canvas

Test case for move generation

Test case for promotion (to queen)

5..3.2. Platform Testing

Software:

- OS – Windows
- Hardware – minimal, enough to run simple chess program
- Software – Python 3.13, Pygame library

5..4. Progression Test Objectives

Ref	Function	Test Objective	Evaluation Criteria	X-Ref	P
1001	Choose_move	AI does not make the same moves every time	AI makes different moves at the beginning of the game	N/A	Medium
1002	negamax	Returns best most in reasonable amount of time	Best move is returned in less than 10 seconds	N/A	High

5..5. Regression Test Objectives

Ref	Function	Test Objective	Evaluation Criteria	X-Ref	P
0007	Get_moves	Returns all moves available for a color	All legal moves in a position are returned	N/A	Medium
0001	N/A	Program does not crash on launch	Program launched successfully without crashing	N/A	High
0008	Choose_move	AI player chooses move of best scoring	Best move out of the move set is returned	N/A	Medium
0002	N/A	Program does not crash when pieces are dragged outside of boundary	Program does not crash when picking up and dragging pieces outside of boundary of game window	N/A	High
0003	Moves_calc	No pieces through each other/illegal moves	Pieces that are unable to jump over pieces (anything that's not a knight) cannot move through other pieces	N/A	High
0004	Is_checkmate	No premature checkmates	The game continues until the king is out of moves	N/A	High
0006	restart	Restart button working	When the game ends, the restart button resets the game	N/A	Low

5..6. Test Strategy

5..6.1. Test level responsibility

Test Level	External Party	Project Team	Business
Unit Testing		P	
Integration Testing		P	
Security Testing		P	
Connectivity Testing		P	
User Acceptance Testing		P	
Production Verification Testing		P	

5..7. Test Type & Approach

Test Type	Objectives
Progression Requirements	Testing will be done after the presentation of the prototyped project. New implementations will function as defined within the test case.
Regression testing	Basic functionality, done before the presentation, does not alter in function before changes and after changes.

5..7.1. Build strategy

The build will be implemented in a one line installation of a git clone command and an execution through navigation to the SRC directory and running python main.py

5..8. Test Execution Schedule

Progression:

- We will play against improved AI several times and ensure that the AI will differ in moves in the same position.

Regression:

- Game launches without crashing
- Move generation will be logged to the console and manually checked
- Promotion will be tested by moving a pawn to the end of the board in PvP mode
- Restart will be tested through functionality of the R key

5..9. Facility, data, and resource provision plan

5..9.1. Test environment

Our game is the testing environment, testing will be done through manual input.

5..9.2. Testing Requirements

Python 3.13

Pygames

Terminal to run the main file

5..10. Testing Tools

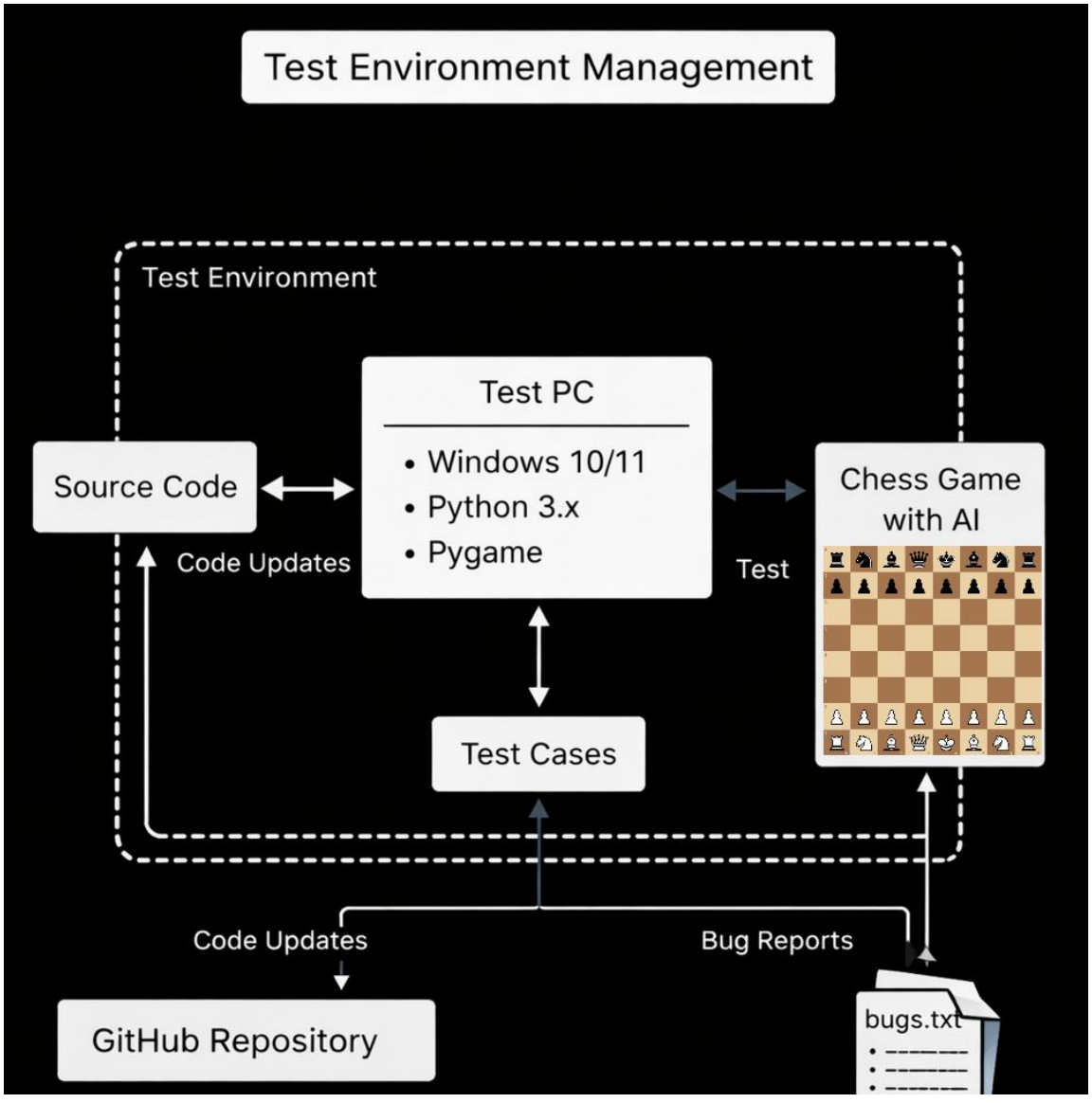
Process	Tool
Test case creation	Word
Test case tracking	Word
Test case execution	Manual
Test case management	Word
Defect management	Not applicable

5..11. Testing Metrics

We will judge on a case by case basis, however, in general, if the program performs as expected, the test will be judged as “passed”, otherwise “fail”

5..12. Test Environment Plan

5..12.1. Test Environment Management



5..13. Test Environment Details

5..13.1. Testers

Vladimir Ryzhov

- Python, Windows OS
- System access not applicable

Kenneth Burke

- Python, Windows OS
- System access not applicable

5..13.2. Hardware and Firmware

Ensure it can run python

5..13.3. Software

Python 3.13

pygame

5..14. Establishing Environment

Task	Requirements	Responsibility	Start Date	End Date
Install Python	Required	All	4/11/2026	4/12/2026
Clone Repository	Required	All	4/11/2026	4/12/2026

5..15. Environment Roles and Responsibilities

Role	Staff Member	Responsibilities
Release Manager	Marc-henry Moise	Responsible for overall establishment, coordination and support of the test environment
Test Manager	Vladimir Ryzhov	Responsible for advising release manager of environment requirements for planning, establishment and ongoing
Project Manager	Kenneth Burke	Escalation point for testing issues.
Tester	Brett Talley	Responsible for test cases

5..16. Assumptions and Dependencies

5..16.1. Assumptions

Testers have knowledge of the codebase, testers know what modules they are testing and how to test for it prior and without training.

5..16.2. Dependencies

Python and Pygame

5..17. Entry and Exit Criteria

Phase 1: Does not crash on launch

- Entry: Launches
- Exit: Game does not crash

Phase 2: Does not crash on canvas boundary issues

- Pieces can be dragged off canvas boundary
- Game does not crash

Phase 3: Piece moves are legal and properly generated

- Legal piece moves show up
- Illegal piece moves do not show up

Phase 4: AI moves are legal

- AI makes legal moves
- AI continues to make legal moves

5..18. Test Milestones and Schedule

Milestone	Planned End Date	Actual End Date	Resource
Phase 1 Criteria	4/12/2026	4/12/2026	
Phase 2 Criteria	4/12/2026	4/12/2026	
Phase 3 Criteria	4/12/2026	4/12/2026	
Phase 4 Criteria	4/12/2026	4/12/2026	

5..19. Test Report

Test ID	Requirement	Pass	Fail	Severity	Details
0001	Lauch successfully	X		N/A	
0002	No boundary crashing		X	Moderate	Dragging pieces and trying to place them outside of left boundary causes the game to crash
0003	No pieces phasing through each other	X		N/A	
0004	No premature checkmate	X		N/A	
0006	Restart buttons working	X		N/A	
0007	Correct moves returned	X		N/A	
0008	AI returns best move	X		N/A	
1001	AI not making the same moves every time	X		N/A	
1002	Negamax functionality	X		N/A	Was able to make moves in less than 10 seconds

5..20. References

#	Document name	Version	Comments
	Assignment instructions	N/A	Reference to how document is filled out.
	The STP template	N/A	Outlines the document

6. Version Control

For our version control, we have utilized Github.

Version control is an important part of any project because it addresses several major potential problems that might arise during the development and offers solutions to them, such as some sort of catastrophic failure which breaks the project. With version control, we can simply roll back any changes and restore project to a previously working state

Github also has useful features such as forks, which allows us to create different branch of our code. Whenever someone in our group needed to make big changes to our program, they could create a fork and work there without fearing that they would be interfering with another person's work. After they were done, they could merge the fork back into main branch.

A smaller but notable feature is the shared "Issues" list, where anybody could create an issue, allowing the group to effectively remember issues, provide comments and suggestions for them, as well as effectively notify the group whenever the issue has been resolved.

7. Conclusion

Overall, I believe we have succeeded in producing a fully functioning chess game, which has the capability to not only be played in Player vs Player mode, but also in Player vs AI mode. The AI offers a good level of challenge to those who choose to play against it.

All the standard chess rules, such as standard piece movement, castling, promotions, and en passant have been implemented as well.